

Lab_7

Luis Gonzalez 23353 - Joel Jaquez 23369 - Fabian Morales 23267

2026-04-17

```
library(dplyr)
```

```
##  
## Adjuntando el paquete: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(readr)  
library(tidyr)  
library(ggplot2)  
library(caret)
```

```
## Cargando paquete requerido: lattice
```

```
library(corrplot)
```

```
## corrplot 0.95 loaded
```

```
load("datos/listings.RData")
```

Actividad 2 - Preeprocesamiento

```
listings_clean_comp <- listings %>%  
  mutate(price = parse_number(price)) %>%  
  select(-c(listing_url, scrape_id, picture_url, host_url,  
            host_thumbnail_url, host_picture_url, description,  
            neighborhood_overview, host_about, amenities))  
  
set.seed(123)  
total_filas <- nrow(listings_clean_comp)
```

```

indice_train <- sample(1:total_filas, size = 0.8 * total_filas)
train_comp <- listings_clean_comp[ indice_train, ]
test_comp <- listings_clean_comp[-indice_train, ]

train_modelo <- train_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

test_modelo <- test_comp %>%
  filter(!is.na(price), price < 1000,
         !is.na(bedrooms), bedrooms <= 10,
         !is.na(bathrooms), !is.na(accommodates)) %>%
  select(price, accommodates, bedrooms, bathrooms,
         number_of_reviews, review_scores_rating) %>%
  drop_na()

cat("Filas entrenamiento:", nrow(train_modelo), "\n")

```

```
## Filas entrenamiento: 48442
```

```
cat("Filas prueba:      ", nrow(test_modelo), "\n")
```

```
## Filas prueba:      12173
```

Para garantizar una comparación válida entre el modelo de Regresión Logística y los algoritmos evaluados en laboratorios anteriores, se reutilizaron exactamente los mismos conjuntos de entrenamiento y prueba generados desde el Laboratorio 3. Utilizando `set.seed(123)` y una división 80/20, el conjunto de entrenamiento quedó con 48,442 observaciones y el de prueba con 12,173, ambos con las mismas 6 variables: `price`, `accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`, aplicando los mismos filtros de precio menor a \$1,000 y habitaciones menores o iguales a 10. Mantener estas condiciones idénticas es fundamental porque cualquier diferencia en los datos de entrenamiento o prueba invalidaría la comparación entre modelos, ya que una mejora en las métricas podría deberse a los datos y no al algoritmo en sí.

Actividad 1 - Variable Dicotómica

```

# Mismos puntos de corte P33=136 y P67=250 del Lab 4 y 5
precios_validos <- listings_clean_comp %>%
  filter(!is.na(price), price > 0, price < 1000) %>%
  pull(price)

p33 <- quantile(precios_validos, 0.33)
p67 <- quantile(precios_validos, 0.67)
cat("P33:", round(p33, 2), "| P67:", round(p67, 2), "\n")

```

```
## P33: 136 | P67: 250
```

```

# Crear variables dicotómicas en train
train_rl <- train_modelo %>%
  mutate(
    es_cara = ifelse(price > p67, 1, 0),
    es_media = ifelse(price > p33 & price <= p67, 1, 0),
    es_barata = ifelse(price <= p33, 1, 0)
  )

# Crear variables dicotómicas en test
test_rl <- test_modelo %>%
  mutate(
    es_cara = ifelse(price > p67, 1, 0),
    es_media = ifelse(price > p33 & price <= p67, 1, 0),
    es_barata = ifelse(price <= p33, 1, 0)
  )

# Verificar distribución
cat("\n--- Distribución en entrenamiento ---\n")

```

```

##
## --- Distribución en entrenamiento ---

```

```

cat("Es cara (1):", sum(train_rl$es_cara),
    "| No cara (0):", sum(train_rl$es_cara == 0), "\n")

```

```

## Es cara (1): 14963 | No cara (0): 33479

```

```

cat("Es media (1):", sum(train_rl$es_media),
    "| No media (0):", sum(train_rl$es_media == 0), "\n")

```

```

## Es media (1): 16690 | No media (0): 31752

```

```

cat("Es barata (1):", sum(train_rl$es_barata),
    "| No barata(0):", sum(train_rl$es_barata == 0), "\n")

```

```

## Es barata (1): 16789 | No barata(0): 31653

```

```

cat("\n--- Distribución en prueba ---\n")

```

```

##
## --- Distribución en prueba ---

```

```

cat("Es cara (1):", sum(test_rl$es_cara),
    "| No cara (0):", sum(test_rl$es_cara == 0), "\n")

```

```

## Es cara (1): 3741 | No cara (0): 8432

```

```
cat("Es media (1):", sum(test_rl$es_media),
    "| No media (0):", sum(test_rl$es_media == 0), "\n")
```

```
## Es media (1): 4265 | No media (0): 7908
```

```
cat("Es barata (1):", sum(test_rl$es_barata),
    "| No barata(0):", sum(test_rl$es_barata == 0), "\n")
```

```
## Es barata (1): 4167 | No barata(0): 8006
```

Se crearon tres variables dicotómicas a partir de los mismos puntos de corte $P33=136$ y $P67=250$ definidos desde el Laboratorio 4, garantizando consistencia con todos los modelos anteriores. La variable `es_cara` toma el valor 1 cuando el precio por noche supera los 250 y 0 en caso contrario, `es_media` vale 1 cuando el precio se encuentra entre 136 y 250, y `es_barata` vale 1 cuando el precio es igual o menor a 136. En el conjunto de entrenamiento, 14,963 propiedades son caras, 16,690 son de precio medio y 16,789 son baratas, mientras que en el conjunto de prueba hay 3,741 caras, 4,265 medias y 4,167 baratas. La distribución es bastante balanceada entre las tres categorías tanto en entrenamiento como en prueba, lo cual es favorable para el entrenamiento de los modelos ya que ninguna clase domina sobre las demás y el modelo tendrá suficientes ejemplos de cada categoría para aprender sus patrones correctamente.

Actividad 3 - Modelo

```
# Actividad 3: Modelo de Regresión Logística - "¿Es cara o no?"
# Variable respuesta: es_cara (1 = cara, 0 = no cara)
# Validación cruzada 10-fold igual que la maestra
```

```
cv_ctrl <- trainControl(method = "cv", number = 10)

set.seed(123)
modelo_cara <- caret::train(
  as.factor(es_cara) ~ accommodates + bedrooms + bathrooms +
    number_of_reviews + review_scores_rating,
  data      = train_rl,
  method    = "glm",
  family    = binomial,
  trControl = cv_ctrl
)

summary(modelo_cara)
```

```
##
## Call:
## NULL
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -6.1163022   0.2123975  -28.80  <2e-16 ***
## accommodates    0.2129436   0.0079481   26.79  <2e-16 ***
## bedrooms       0.2069022   0.0189549   10.91  <2e-16 ***
```

```
## bathrooms          0.7388290  0.0212645   34.74   <2e-16 ***
## number_of_reviews  -0.0035004  0.0001601  -21.87   <2e-16 ***
## review_scores_rating 0.6000963  0.0435503   13.78   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 59894 on 48441 degrees of freedom
## Residual deviance: 46174 on 48436 degrees of freedom
## AIC: 46186
##
## Number of Fisher Scoring iterations: 5
```

```
cat("\nAIC:", AIC(modelo_cara$finalModel), "\n")
```

```
##
## AIC: 46185.63
```

```
cat("BIC:", BIC(modelo_cara$finalModel), "\n")
```

```
## BIC: 46238.36
```

Se elaboró un modelo de Regresión Logística con validación cruzada de 10 pliegues para predecir si una propiedad es cara o no, usando las 5 variables predictoras numéricas disponibles. El resumen del modelo muestra que todas las variables son altamente significativas ($p < 2e-16$ en todos los casos), lo que indica que cada una aporta información relevante para determinar si una propiedad pertenece al segmento caro. Los coeficientes positivos de bathrooms (0.7388), review_scores_rating (0.6001), accommodates (0.2129) y bedrooms (0.2069) indican que a mayor valor de estas variables, mayor es la probabilidad de que una propiedad sea cara, lo cual tiene sentido lógico: propiedades más grandes y mejor calificadas tienden a tener precios más altos. Por el contrario, number_of_reviews tiene un coeficiente negativo (-0.0035), sugiriendo que propiedades con muchas reseñas tienden a ser menos caras, posiblemente porque son propiedades más accesibles con mayor rotación de huéspedes. El AIC del modelo es 46,185.63 y el BIC 46,238.36, valores que servirán de referencia para comparar con modelos posteriores.

```
# Predicción con umbral 0.5
pred_prob_cara <- predict(modelo_cara,
                          newdata = test_rl,
                          type     = "prob")[, "1"]

pred_cara <- ifelse(pred_prob_cara >= 0.5, 1, 0)

cm_cara <- confusionMatrix(as.factor(pred_cara),
                           as.factor(test_rl$es_cara),
                           positive = "1")
print(cm_cara)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 7613 1925
```

```
##          1   819 1816
##
##          Accuracy : 0.7746
##          95% CI : (0.7671, 0.782)
##    No Information Rate : 0.6927
##    P-Value [Acc > NIR] : < 2.2e-16
##
##          Kappa : 0.4231
##
##    McNemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.4854
##          Specificity : 0.9029
##    Pos Pred Value : 0.6892
##    Neg Pred Value : 0.7982
##          Prevalence : 0.3073
##    Detection Rate : 0.1492
##    Detection Prevalence : 0.2165
##    Balanced Accuracy : 0.6942
##
##    'Positive' Class : 1
##
```

Evaluado sobre el conjunto de prueba con umbral de decisión en 0.5, el modelo alcanzó un Accuracy de 77.46% y un Kappa de 0.4231, lo que representa una concordancia moderada más allá del azar. La Especificidad es alta con 90.29%, lo que significa que el modelo identifica correctamente el 90% de las propiedades que no son caras. Sin embargo, la Sensibilidad es de apenas 48.54%, indicando que solo clasifica correctamente la mitad de las propiedades que sí son caras. Esto sugiere que el umbral de 0.5 genera un modelo conservador que prefiere predecir “no cara” ante la duda, lo cual se refleja en que de 3,741 propiedades caras reales, el modelo solo identificó correctamente 1,816 y dejó escapar 1,925.

Actividad 4 - Correlacion

```
# Actividad 4: Correlación entre predictores
```

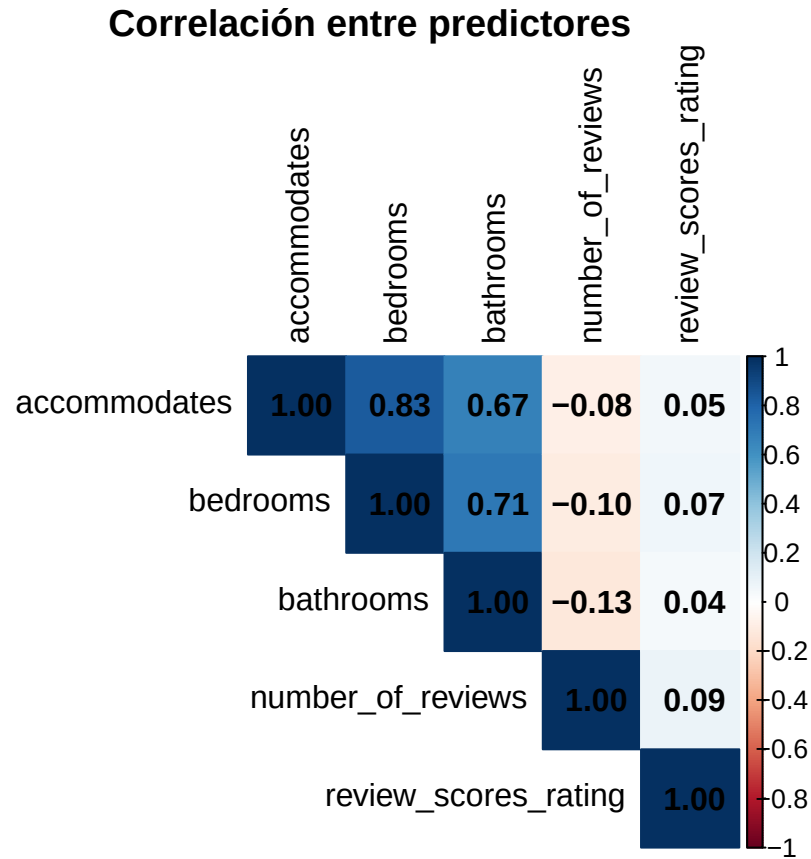
```
library(corrplot)
```

```
cor_matrix <- cor(train_rl %>%
  select(accommodates, bedrooms, bathrooms,
    number_of_reviews, review_scores_rating))
print(round(cor_matrix, 3))
```

```
##          accommodates bedrooms bathrooms number_of_reviews
## accommodates          1.000    0.831    0.672          -0.083
## bedrooms              0.831    1.000    0.713          -0.102
## bathrooms             0.672    0.713    1.000          -0.130
## number_of_reviews     -0.083   -0.102   -0.130           1.000
## review_scores_rating    0.054    0.070    0.042           0.086
##
##          review_scores_rating
## accommodates          0.054
## bedrooms              0.070
## bathrooms             0.042
```

```
## number_of_reviews      0.086
## review_scores_rating   1.000
```

```
corrplot(cor_matrix,
  method      = "color",
  type        = "upper",
  addCoef.col = "black",
  tl.col      = "black",
  title       = "Correlación entre predictores",
  mar         = c(0,0,1,0))
```



```
# VIF para detectar multicolinealidad
# VIF > 5 indica problema, VIF > 10 es severo
library(car)
```

```
## Cargando paquete requerido: carData
```

```
##
```

```
## Adjuntando el paquete: 'car'
```

```
## The following object is masked from 'package:dplyr':
```

```
##
```

```
## recode
```

```
vif(modelo_cara$finalModel)
```

```
##          accommodates          bedrooms          bathrooms
##          2.552220          2.859127          1.616471
##    number_of_reviews review_scores_rating
##          1.015063          1.007474
```

El análisis de multicolinealidad mediante el Factor de Inflación de Varianza (VIF) muestra que ninguna variable presenta valores problemáticos: accommodates (2.55), bedrooms (2.86), bathrooms (1.62), number_of_reviews (1.02) y review_scores_rating (1.01), todos muy por debajo del umbral de 5 que indicaría multicolinealidad moderada y lejos del valor de 10 que sería severo. Esto confirma que aunque las variables están relacionadas entre sí, ninguna está siendo explicada por las demás de forma que afecte la estabilidad del modelo.

El mapa de correlación complementa este análisis y muestra que la relación más fuerte entre predictores es entre accommodates y bedrooms (0.83), lo que tiene sentido ya que propiedades con más habitaciones generalmente alojan más personas. También existe una correlación moderada entre bathrooms y accommodates (0.67) y entre bathrooms y bedrooms (0.71). Sin embargo, estas correlaciones no son suficientemente altas como para generar multicolinealidad problemática, lo cual queda confirmado por los valores de VIF. Las variables number_of_reviews y review_scores_rating prácticamente no tienen correlación con el resto de predictores, lo que indica que aportan información completamente independiente al modelo. En conclusión, el modelo está bien construido, no presenta problemas de multicolinealidad y todas sus variables son significativas y aportan información relevante para predecir si una propiedad es cara o no.

Actividad 5: Conjunto de prueba.

```
acc <- cm_cara$overall["Accuracy"]
kappa <- cm_cara$overall["Kappa"]
sens <- cm_cara$byClass["Sensitivity"]
spec <- cm_cara$byClass["Specificity"]
ppv <- cm_cara$byClass["Pos Pred Value"] # Precisión
npv <- cm_cara$byClass["Neg Pred Value"]
f1 <- cm_cara$byClass["F1"]
bacc <- cm_cara$byClass["Balanced Accuracy"]

# AUC - ROC
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
```

```
## Adjuntando el paquete: 'pROC'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      cov, smooth, var
```

```
roc_cara <- roc(test_rl$es_cara, pred_prob_cara, quiet = TRUE)
auc_val <- as.numeric(auc(roc_cara))
```

```
cat("----- Eficiencia en conjunto de prueba -----\\n")
```

```
## ----- Eficiencia en conjunto de prueba -----
```

```
cat("Accuracy      :", round(acc, 4), "\n")
```

```
## Accuracy      : 0.7746
```

```
cat("Kappa         :", round(kappa,4), "\n")
```

```
## Kappa         : 0.4231
```

```
cat("Sensibilidad   :", round(sens, 4), "\n")
```

```
## Sensibilidad   : 0.4854
```

```
cat("Especificidad  :", round(spec, 4), "\n")
```

```
## Especificidad  : 0.9029
```

```
cat("Precisión (PPV) :", round(ppv, 4), "\n")
```

```
## Precisión (PPV) : 0.6892
```

```
cat("NPV           :", round(npv, 4), "\n")
```

```
## NPV           : 0.7982
```

```
cat("F1-Score      :", round(f1, 4), "\n")
```

```
## F1-Score      : 0.5696
```

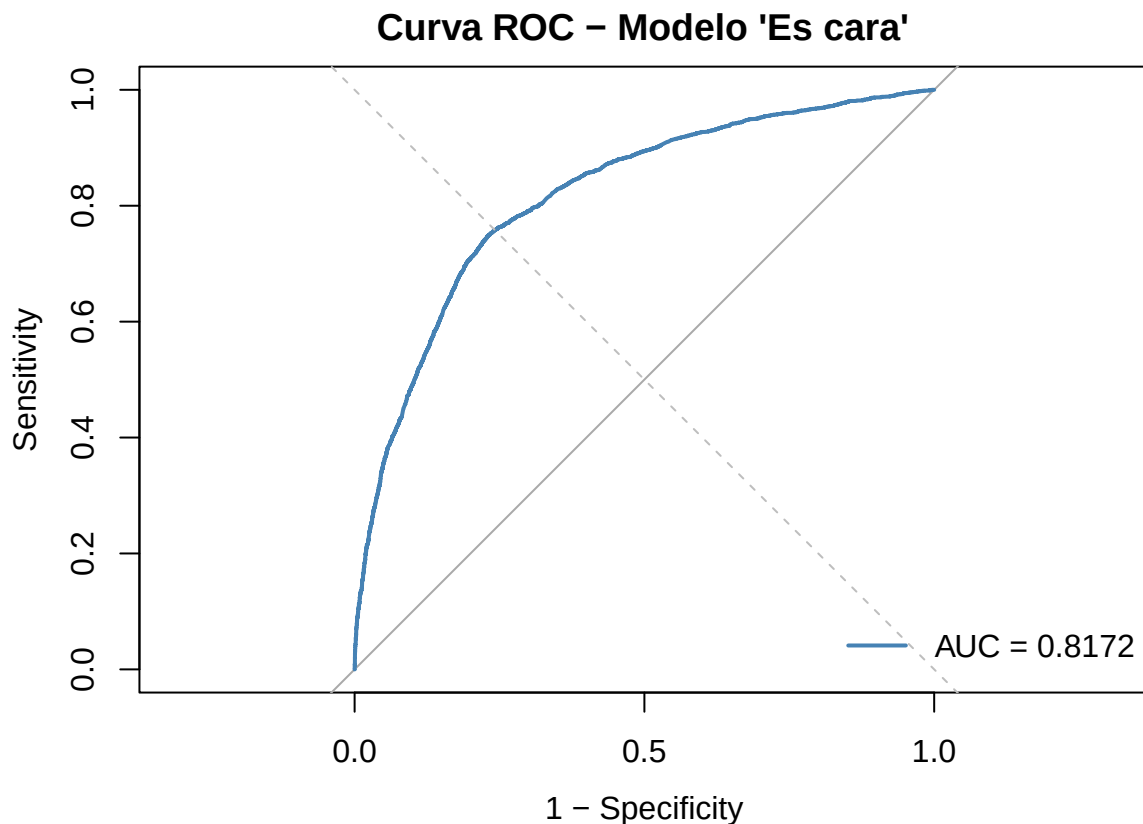
```
cat("Balanced Accuracy:", round(bacc, 4), "\n")
```

```
## Balanced Accuracy: 0.6942
```

```
cat("AUC           :", round(auc_val, 4), "\n")
```

```
## AUC           : 0.8172
```

```
plot(roc_cara, main = "Curva ROC - Modelo 'Es cara'",  
     col = "steelblue", lwd = 2, legacy.axes = TRUE)  
abline(a = 0, b = 1, lty = 2, col = "gray")  
legend("bottomright", legend = paste("AUC =", round(auc_val, 4)),  
      col = "steelblue", lwd = 2, bty = "n")
```



La curva ROC obtenida confirma visualmente lo anticipado por las métricas numéricas. La curva se aleja claramente de la diagonal de referencia y se acerca a la esquina superior izquierda, lo cual indica buena capacidad discriminativa del modelo. El AUC de 0.8172 se interpreta como que, si tomamos al azar una propiedad cara y una no cara, el modelo le asignará una probabilidad mayor a la cara el 81.72% de las veces.

Actividad 6

```
# Predicción y error
pred_prob_train <- predict(modelo_cara, newdata = train_rl, type = "prob")[, "1"]
pred_train      <- ifelse(pred_prob_train >= 0.5, 1, 0)
error_train     <- mean(pred_train != train_rl$es_cara)
acc_train       <- 1 - error_train

error_test <- mean(pred_cara != test_rl$es_cara)
acc_test  <- 1 - error_test

acc_cv    <- modelo_cara$results$Accuracy
error_cv  <- 1 - acc_cv

cat("----- Comparación de errores -----<n")

## ----- Comparación de errores -----
```

```
cat("Accuracy entrenamiento :", round(acc_train, 4),
    "| Error:", round(error_train, 4), "\n")
```

```
## Accuracy entrenamiento : 0.7764 | Error: 0.2236
```

```
cat("Accuracy validación CV :", round(acc_cv, 4),
    "| Error:", round(error_cv, 4), "\n")
```

```
## Accuracy validación CV : 0.7762 | Error: 0.2238
```

```
cat("Accuracy prueba :", round(acc_test, 4),
    "| Error:", round(error_test, 4), "\n")
```

```
## Accuracy prueba : 0.7746 | Error: 0.2254
```

```
cat("Diferencia train - test:", round(acc_train - acc_test, 4), "\n")
```

```
## Diferencia train - test: 0.0019
```

```
# Curvas de aprendizaje.
set.seed(123)
tamanos <- seq(0.1, 1.0, by = 0.1)
err_train_v <- numeric(length(tamanos))
err_test_v <- numeric(length(tamanos))

for (i in seq_along(tamanos)) {
  n_sub <- floor(tamanos[i] * nrow(train_rl))
  idx <- sample(1:nrow(train_rl), n_sub)
  sub <- train_rl[idx, ]

  mod_tmp <- glm(es_cara ~ accommodates + bedrooms + bathrooms +
                  number_of_reviews + review_scores_rating,
                  data = sub, family = binomial)

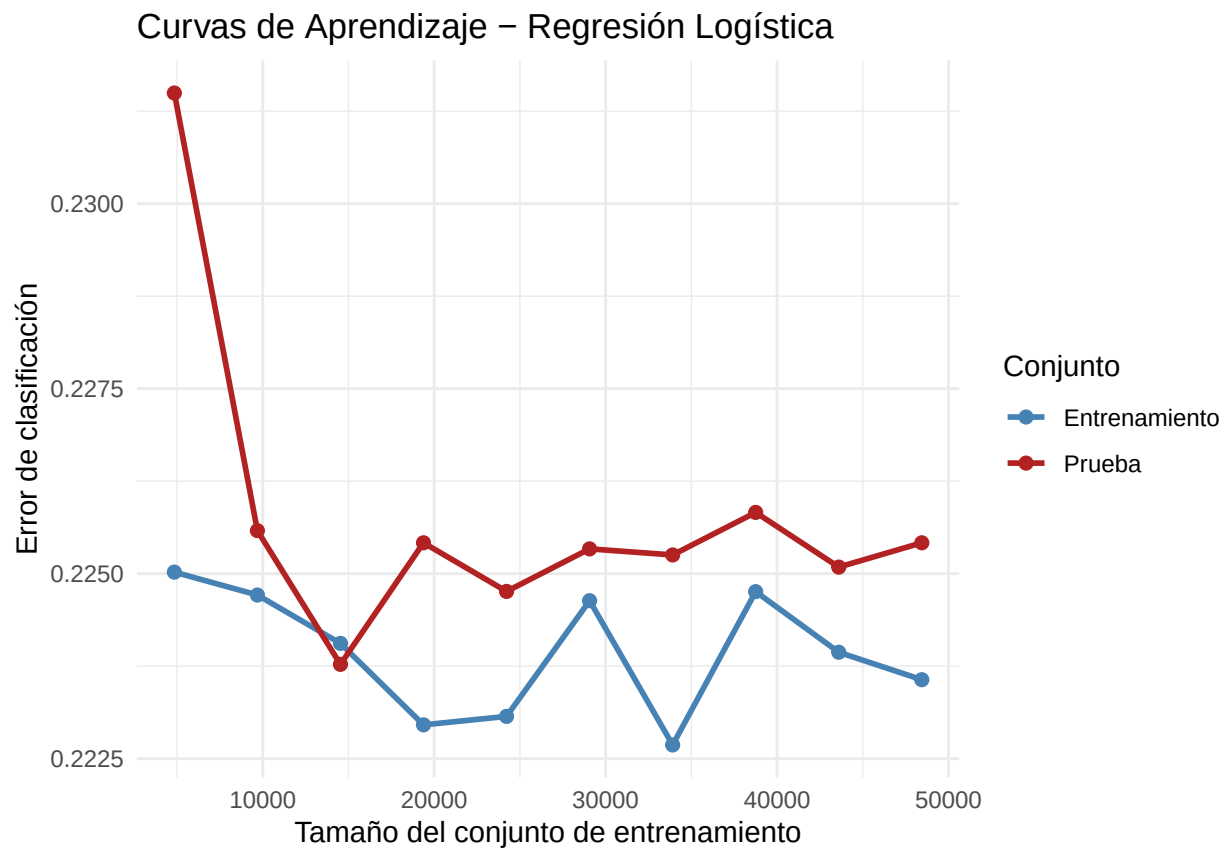
  p_tr <- ifelse(predict(mod_tmp, sub, type = "response") >= 0.5, 1, 0)
  err_train_v[i] <- mean(p_tr != sub$es_cara)

  p_te <- ifelse(predict(mod_tmp, test_rl, type = "response") >= 0.5, 1, 0)
  err_test_v[i] <- mean(p_te != test_rl$es_cara)
}

df_curvas <- data.frame(
  Tamano = tamanos * nrow(train_rl),
  Entrenamiento = err_train_v,
  Prueba = err_test_v
) %>%
  pivot_longer(cols = c(Entrenamiento, Prueba),
               names_to = "Conjunto", values_to = "Error")
```

```
ggplot(df_curvas, aes(x = Tamano, y = Error, color = Conjunto)) +
  geom_line(size = 1) +
  geom_point(size = 2) +
  labs(title = "Curvas de Aprendizaje - Regresión Logística",
       x = "Tamaño del conjunto de entrenamiento",
       y = "Error de clasificación") +
  theme_minimal() +
  scale_color_manual(values = c("Entrenamiento" = "steelblue",
                                "Prueba" = "firebrick"))
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once per session.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



La gráfica de curvas de aprendizaje confirma de manera contundente que el modelo de Regresión Logística no presenta sobreajuste. Tanto el error de entrenamiento (línea azul) como el error de prueba (línea roja) convergen rápidamente a un valor muy similar en torno al 22.3-22.5%, y la separación entre ambas curvas es mínima.